

Linux Networking Commands Reference

1. ip - Modern network configuration (replaces ifconfig/route/arp)

Description: The all-in-one utility for viewing and configuring interfaces, IP addresses, routing tables, and neighbor (ARP) tables. It is the current standard on almost all modern Linux distributions.

Use case: A student sets up a virtual machine and needs to manually assign it a static IP address and default gateway so it can reach the internet, instead of relying on DHCP.

Structure: ip [OPTIONS] OBJECT COMMAND

Common objects: addr, link, route, neigh, rule

Command	Purpose
ip addr show	Show all IP addresses
ip addr add 192.168.1.10/24 dev eth0	Assign IP to interface
ip addr del 192.168.1.10/24 dev eth0	Remove IP from interface
ip link show	List network interfaces
ip link set eth0 up / down	Enable/disable interface
ip link set eth0 mtu 1400	Change MTU
ip route show	Show routing table
ip route add default via 192.168.1.1	Add default gateway
ip route del 10.0.0.0/8	Delete a route
ip neigh show	Show ARP/neighbor table
ip -s link show eth0	Show interface statistics

Example:

```
ip addr add 10.0.0.5/24 dev eth0
ip link set eth0 up
ip route add default via 10.0.0.1
```

2. ifconfig - Legacy interface configuration (deprecated but still common)

Description: The older, pre-ip command for viewing and configuring network interfaces. It's deprecated on many distros but still widely taught and found in older scripts and exam material.

Use case: A student working through an older lab manual or textbook is asked to bring up an interface and assign it an IP the "classic" way, before learning the modern ip equivalent.

Structure: ifconfig [interface] [options]

Option	Purpose
ifconfig	Show all active interfaces
ifconfig -a	Show all interfaces (including down)
ifconfig eth0 up / down	Enable/disable interface
ifconfig eth0 192.168.1.10 netmask 255.255.255.0	Assign IP
ifconfig eth0 mtu 1400	Set MTU

Example:

```
ifconfig eth0 192.168.1.10 netmask 255.255.255.0 up
```

3. ping -Test host reachability

Description: Sends ICMP Echo Request packets to a target host and reports whether replies come back, along with round-trip time. It's the most basic "is the network up?" test.

Use case: A student's application can't reach a server, so the first troubleshooting step is to ping the server's IP to confirm basic network connectivity before checking anything else.

Structure: ping [options] destination

Option	Purpose
-c N	Send N packets then stop
-i N	Interval between packets (seconds)
-s N	Packet size in bytes
-t N (or -ttl N)	Set TTL
-W N	Timeout for response (seconds)
-4 / -6	Force IPv4/IPv6

Example:

```
ping -c 4 8.8.8.8
```

4. traceroute / tracepath 0- Trace packet route to host

Description: Shows every router (hop) a packet passes through on its way to a destination, along with the time taken at each hop. Useful for pin pointing where along a path a delay or failure occurs.

Use case: A student notices a website loads slowly and uses traceroute to find out whether the delay happens near their own router, their ISP, or somewhere further along the path to the server.

Structure: traceroute [options] destination

Option	Purpose
-n	Don't resolve hostnames
-m N	Max hops
-p N	Destination port
-I	Use ICMP instead of UDP
-T	Use TCP SYN

Example:

```
traceroute -n google.com
```

5. netstat - Network connections, routing, interface stats (legacy)

Description: Displays active network connections, listening ports, and routing/interface statistics. It's the older tool for this job; **ss** is its modern replacement, but netstat is still very common in tutorials.

Use case: A student runs a web server locally and wants to confirm it's actually listening on port 80 before trying to access it from a browser.

Structure: netstat [options]

Option	Purpose
-t	TCP connections
-u	UDP connections
-l	Listening sockets only
-n	Numeric addresses (no DNS lookup)

-p	Show PID/program name
-r	Show routing table
-i	Show interface stats

Example:

```
netstat -tulnp
```

6. ss - Modern socket statistics (replaces netstat)

Description: A faster, modern replacement for netstat that reads socket information directly from the kernel. It shows the same kind of connection/listening-port information but performs better on systems with many connections.

Use case: A student is debugging why two services on the same machine can't talk to each other and uses ss to check whether the expected port is actually open and listening.

Structure: ss [options] [filter]

Option	Purpose
-t	TCP sockets
-u	UDP sockets
-l	Listening sockets
-n	Numeric (no name resolution)
-p	Show process using socket
-a	All sockets (listening + established)
-s	Summary statistics

Example:

```
ss -tulnp
ss -t state established
```

7. dig - DNS lookup (detailed)

Description: Queries DNS servers directly and returns detailed information about a domain's records (A, MX, NS, TXT, etc.). It's the preferred tool for serious DNS troubleshooting because of its detailed, script-friendly output.

Use case: A student sets up email hosting for a class project and uses dig to verify that the domain's MX records actually point to the right mail server.

Structure: dig [@server] name [type] [options]

Option	Purpose
+short	Short output (just the answer)
-x IP	Reverse DNS lookup
@server	Query a specific DNS server
MX / NS / TXT / A / AAAA	Record type
+trace	Trace full resolution path

Example:

- **dig** – Domain Information Groper, a DNS lookup utility.
- **google.com** – The domain name being queried.
- **A** – Request the IPv4 address (A record).
- **+short** – Display only the answer without the detailed DNS information.

```
dig google.com A +short
dig @8.8.8.8 example.com MX
```

Sample Output:

142.251.221.174

What this tells us: Your system successfully:

Contacted a DNS server, Resolved google.com into an IPv4 address, Received the address 142.251.221.174.

8. nslookup - Simple DNS lookup

Description: A simpler, older DNS lookup tool. It gives quicker, less detailed output than dig and is often the first DNS tool students learn because of its straightforward syntax.

Use case: A student wants a quick check of what IP address a domain resolves to, without needing all the extra detail dig provides.

Structure: nslookup [name] [server]

Example:

```
nslookup google.com
nslookup google.com 8.8.8.8
```

9. host - Quick DNS lookup utility

Description: A minimal DNS lookup tool designed for quick, script-friendly one-line answers, sitting between nslookup's simplicity and dig's detail.

Use case: A student writing a shell script needs a domain's IP address in a single clean line of output to feed into another command.

Structure: host [options] name [server]

Option	Purpose
-t TYPE	Query specific record type
-a	Show all record types

Example:

```
host -t MX example.com
```

asks the DNS for the MX (Mail Exchanger) record of example.com.

Sample Output: example.com mail is handled by 0 .

What does this mean:

MX record: Specifies which mail server accepts email for a domain.

Priority 0: The preference value. Lower values have higher priority.

. (root domain): Indicates that there is no mail server for this domain.

In other words, example.com is configured to not accept email.

10. curl - Transfer data / test HTTP(S) endpoints

Description: Transfers data to or from a server using protocols like HTTP, HTTPS, and FTP. It's the go-to tool for testing APIs and web servers directly from the command line, showing exactly what a server returns.

Use case: A student building a REST API tests whether a POST endpoint correctly accepts JSON data before wiring it up to a front-end application.

Structure: curl [options] URL

Option	Purpose
-I	Fetch headers only
-X METHOD	Specify HTTP method (GET, POST...)
-d DATA	Send POST data
-H "Header: val"	Add a custom header
-o file	Save output to file
-L	Follow redirects
-v	Verbose (debug) output
-k	Ignore SSL certificate errors

Example:

```
curl -X POST -H "Content-Type: application/json" \
-d '{"key":"val"}' https://api.example.com/data
```

Part	Meaning
curl	Command-line tool used to transfer data to or from a server using HTTP, HTTPS, FTP, etc.
-X POST	Specifies that the HTTP method is POST . POST is typically used to create or submit data to a server.
-H "Content-Type: application/json"	Adds an HTTP header indicating that the request body is in JSON format.
-d '{"key":"val"}'	Sends the request body (payload). Here, the JSON object contains one key-value pair: "key": "val".
https://api.example.com/data	The URL (endpoint) where the request is sent.

What the HTTP request looks like, when curl sends this request, it is similar to:

POST /data HTTP/1.1

Host: api.example.com

Content-Type: application/json

Content-Length: 13

{"key":"val"}

➔ Request Method: POST, Headers: Tell the server about the request. Body: Contains the JSON data.

Real-world example

Suppose an API accepts information about a student.

**curl -X POST **

**-H "Content-Type: application/json" **

-d '{

```
1. "name":"Rahul",
2. "age":22,
3. "course":"Embedded Systems"
}\
```

https://college.example.edu/api/students

The server receives:

```
{
  "name": "Rahul",
  "age": 22,
  "course": "Embedded Systems"
}
```

and may respond with:

```
{
  "id": 101,
  "status": "Student created"
}
```

11. wget - Download files over network

Description: Downloads files from the web over HTTP, HTTPS, or FTP. Unlike curl, it's built specifically for saving files to disk and supports resuming interrupted downloads and recursive site downloads.

Use case: A student downloading a large dataset for a machine learning assignment loses their connection partway through and resumes the download instead of starting over.

Structure: `wget [options] URL`

Option	Purpose
-O file	Save with specific filename
-c	Resume interrupted download
-r	Recursive download
-q	Quiet mode
--limit-rate=100k	Limit bandwidth

Example: Download the GNU GPL license text

```
wget -O license.txt https://raw.githubusercontent.com/EbookFoundation/free-programming-books/main/LICENSE
```

12. ssh - Secure remote login

Description: Opens an encrypted terminal session on a remote machine, allowing commands to be run as if sitting in front of it. It's the standard way to administer Linux servers remotely.

Use case: A student needs to run code on a university lab server or cloud VM from their laptop, so they SSH into it to get a command-line session.

Structure: `ssh [options] [user@]host [command]`

Option	Purpose
-p N	Port number
-i file	Identity/private key file
-L local:host:remote	Local port forwarding
-R remote:host:local	Remote port forwarding
-D port	Dynamic (SOCKS) proxy
-v	Verbose (debugging)

Example: I am currently in the account Raghu and want to switch to account/system having user name hamproject

```
raghu@ishpc:~/raghu/Mango_leaf_disease_classification$ ssh hamproject@172.16.18.235
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
hamproject@172.16.18.235's password:*****
hamproject@ishpc:~$
```

13. scp - Secure copy over SSH

Description: Copies files between machines over an encrypted SSH connection. It's essentially "cp" but for a remote host, and is one of the simplest ways to move files securely.

Use case: A student finishes an assignment locally and copies the finished project files to a remote server for submission or deployment.

Structure: scp [options] source destination

Option	Purpose
-P N	Port number
-r	Recursive (directories)
-i file	Identity key file

Example:

```
raghu@ishpc:~/raghu/Mango_leaf_disease_classification$ scp
/home/raghu/raghu/Mango_leaf_disease_classification/license.txt \
    hamproject@ishpc:/home/hamproject/hamproject/
Are you sure you want to continue connecting (yes/no/[fingerprint])? Yes
hamproject@ishpc's password:
license.txt                                100% 18KB 28.5MB/s 00:00
raghu@ishpc:~/raghu/Mango_leaf_disease_classification$ ssh hamproject@172.16.18.235
hamproject@ishpc:~$ cd hamproject
hamproject@ishpc:~/hamproject$ pwd
/home/hamproject/hamproject
hamproject@ishpc:~/hamproject$ ls
```

license.txt should be present

14. rsync - Efficient file synchronization

Description: Synchronizes files and directories between locations, copying only the differences instead of everything each time. This makes it far faster than scp for repeated backups or large directory trees.

Use case: A student backs up their project folder to a remote server every night; after the first full copy, rsync only transfers the files that actually changed.

Structure: rsync [options] source destination

Option	Purpose
-a	Archive mode (preserves permissions, etc.)
-v	Verbose
-z	Compress during transfer
-e "ssh -p N"	Use SSH with custom port
--delete	Delete extraneous files at destination

Example:

```
rsync -avz -e "ssh -p 2222" /local/dir/ user@remote:/remote/dir/
```

15. nmap - Network/port scanning

Description: Scans hosts and networks to discover which machines are alive, which ports are open, and what services/versions are running on them. Widely used in security auditing and network inventory.

Use case: In a networking lab, a student scans their own virtual lab network to see which of their VMs are reachable and which ports are unintentionally left open.

Structure: nmap [options] target

Option	Purpose
-sS	TCP SYN (stealth) scan
-sV	Detect service versions
-O	Detect OS
-p N or -p 1-1000	Specify port(s)
-A	Aggressive scan (OS, version, scripts)
-Pn	Skip host discovery (treat as up)

Example:

```
nmap 192.168.1.10
```

```
Starting Nmap 7.80 ( https://nmap.org ) at 2026-07-29 11:52 IST
Nmap scan report for ishpc.ac.in (172.16.18.235)
Host is up (0.0010s latency).
Not shown: 988 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
443/tcp   open  https
```



```
3000/tcp open  ppp
5000/tcp open  upnp
5050/tcp open  mmcc
5432/tcp open  postgresql
8080/tcp open  http-proxy
9000/tcp open  cslistener
9090/tcp open  zeus-admin
9091/tcp open  xmltec-xmlmail
9100/tcp open  jetdirect
```

Nmap done: 1 IP address (1 host up) scanned in 0.14 seconds

Note: only scan networks/hosts you own or have explicit permission to test.

Command	Purpose
nmap 192.168.1.10	Scan a single host
nmap example.com	Scan a hostname
nmap -sn 192.168.1.0/24	Discover live hosts
nmap -p 22,80,443 host	Scan specific ports
nmap -sV host	Detect service versions
sudo nmap -O host	Detect operating system
sudo nmap -A host	Aggressive scan
nmap -p- host	Scan all TCP ports
sudo nmap -sU host	Scan UDP ports
nmap -oN results.txt host	Save results to a text file

16. tcpdump - Packet capture and analysis

Description: Captures live network traffic on an interface and displays or saves the raw packets, so you can see exactly what's being sent and received at the protocol level.

Use case: A student learning about the TCP three-way handshake captures live traffic while connecting to a web server to actually see the SYN, SYN-ACK, and ACK packets in order.

Structure: tcpdump [options] [filter expression]

Option	Purpose
-i eth0	Capture on specific interface
-n	Don't resolve hostnames
-w file.pcap	Write capture to file
-r file.pcap	Read from capture file
-c N	Capture N packets then stop
-X	Show packet contents in hex/ASCII

Example:

```
tcpdump -D #List all network interfaces
```

Output:

```
1.ens27f0np0 [Up, Running, Connected]
```

```
2.ens27f1np1 [Up, Running, Connected]
3.br-2a353a6e1789 [Up, Running, Connected]
4.docker0 [Up, Running, Connected]
5.veth7dc89d3 [Up, Running, Connected]
6.veth0e5074b [Up, Running, Connected]
7.vetha0362ca [Up, Running, Connected]
8.veth13cf984 [Up, Running, Connected]
9.veth9f3debe [Up, Running, Connected]
10.veth72f5d63 [Up, Running, Connected]
11.veth76b2733 [Up, Running, Connected]
12.vethf9cc4c5 [Up, Running, Connected]
13.veth008b519 [Up, Running, Connected]
14.vethd4883fb [Up, Running, Connected]
15.any (Pseudo-device that captures on all interfaces) [Up, Running]
16.lo [Up, Running, Loopback]
17.br-184f8956038c [Up, Disconnected]
18.br-fff960e21284 [Up, Disconnected]
19.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
20.nflog (Linux netfilter log (NFLOG) interface) [none]
21.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
22.dbus-system (D-Bus system bus) [none]
23.dbus-session (D-Bus session bus) [none]
```

Now Capture only 10 packets on a specific interface

```
sudo tcpdump -i ens27f0np0 -c 10
```

output:

[sudo] password for raghu:

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode

listening on ens27f0np0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

11:58:56.639746 IP ishpc.ac.in.https > 172.16.73.35.62839: Flags [P.], seq 3618348236:3618348446, ack 1481956636, win 637, length 210

11:58:56.648030 IP6 fe80::eab5:d0ff:fe3a:acf > ff02::1:ff83:5ae0: ICMP6, neighbor solicitation, who has fe80::544c:7324:e883:5ae0, length 32

11:58:56.712471 IP6 fe80::5422:5837:78be:3158.dhcpv6-client > ff02::1:2.dhcpv6-server: dhcp6 inf-req

11:58:56.725759 IP ishpc.ac.in.42632 > mpl-dc-adc01.manipal.edu.domain: 18483+ [1au] PTR? 35.73.16.172.in-addr.arpa. (54)

11:58:56.734502 IP ishpc.ac.in.https > c-69-143-185-128.hsd1.md.comcast.net.48656: Flags [R], seq 1860930823, win 0, length 0

11:58:56.734554 IP ishpc.ac.in.https > c-69-143-185-128.hsd1.md.comcast.net.48656: Flags [R], seq 1860930823, win 0, length 0

11:58:56.743690 STP 802.1d, Config, Flags [none], bridge-id 8012.78:64:a0:77:d3:00.8063, length 35

11:58:56.790930 IP MAHEFATYL2023.manipal.edu.mdns > mdns.mcast.net.mdns: 0 A (QM)?

mahefatyl1117.local. (37)

11:58:56.791591 IP MAHEFATYL2023.manipal.edu.mdns > mdns.mcast.net.mdns: 0 AAAA (QM)?

mahefatyl1117.local. (37)

11:58:56.801420 IP ishpc.ac.in.42632 > mpl-dc-adc01.manipal.edu.domain: 18483+ PTR? 35.73.16.172.in-addr.arpa. (43)

10 packets captured

185 packets received by filter

133 packets dropped by kernel

17. route - Legacy routing table management

Description: The older tool for viewing and editing the kernel's IP routing table, superseded by `ip route` but still seen in older documentation.

Use case: A student following an older networking course adds a static route so traffic to a specific subnet goes through a particular gateway.

Structure: `route [options]`

Command	Purpose
<code>route -n</code>	Show routing table (numeric)
<code>route add default gw 192.168.1.1</code>	Add default gateway
<code>route del -net 10.0.0.0/8</code>	Delete a route

18. arp - Legacy ARP table management

Description: Displays and edits the ARP (Address Resolution Protocol) table, which maps IP addresses to MAC addresses on the local network. Superseded by `ip neigh` but still commonly referenced.

Use case: A student suspects an IP address conflict on the lab network and checks the ARP table to see which MAC address is currently associated with a given IP.

Structure: `arp [options]`

Command	Purpose
<code>arp -a</code>	Show ARP table
<code>arp -s IP MAC</code>	Add static ARP entry
<code>arp -d IP</code>	Delete ARP entry

19. hostname / hostnamectl — Manage system hostname

Description: Displays or sets the system's hostname — the human-readable name a machine identifies itself by on the network.

Use case: A student sets up several VMs for a clustering exercise and gives each one a distinct hostname so they're easy to tell apart when connecting over SSH.

Structure: `hostname [new-name]` or `hostnamectl [command]`

Example:

```
hostname
hostnamectl set-hostname server01
```

20. nmcli - NetworkManager command-line tool

Description: The command-line interface to NetworkManager, the service that manages network connections (Wi-Fi, Ethernet, VPN) on most desktop Linux distributions.

Use case: A student on a laptop without a graphical desktop needs to connect to campus Wi-Fi entirely from the terminal.

Structure: `nmcli [object] [command]`

Command	Purpose
<code>nmcli device status</code>	Show device status
<code>nmcli connection show</code>	List connections
<code>nmcli device wifi list</code>	Scan Wi-Fi networks

nmcli connection up "SSID"	Connect to network
nmcli connection add type ethernet ifname eth0 ip4 10.0.0.5/24 gw4 10.0.0.1	Add static connection

21. iptables - Legacy firewall/packet filtering

Description: Configures the Linux kernel's packet-filtering rules (firewall), controlling which traffic is allowed, blocked, or redirected. Being replaced by nftables on newer distros, but still the most widely taught firewall tool.

Use case: A student securing a server allows SSH traffic but blocks all other incoming connections, so only remote administration is possible from outside.

Structure: iptables [-t table] COMMAND chain rule-specification -j target

Option	Purpose
-A chain	Append rule
-D chain	Delete rule
-L	List rules
-p protocol	Protocol (tcp/udp/icmp)
--dport N	Destination port
-j ACCEPT/DROP/REJECT	Action (target)

Example:

```
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -A INPUT -p tcp --dport 80 -j DROP
iptables -L -n -v
```

22. ufw — Uncomplicated Firewall (Debian/Ubuntu front-end for iptables)

Description: A simplified, beginner-friendly front-end for iptables on Ubuntu/Debian systems, letting you manage firewall rules with plain-English commands instead of raw iptables syntax.

Use case: A student setting up their first Ubuntu server wants basic firewall protection without needing to learn the full complexity of iptables syntax right away.

Structure: ufw [command]

Command	Purpose
ufw enable / disable	Turn firewall on/off
ufw allow 22/tcp	Allow a port
ufw deny 80/tcp	Block a port
ufw status verbose	Show current rules

Example:

```
ufw allow 22/tcp
ufw enable
```

23. mtr — Combined ping + traceroute (live diagnostics)

Description: Combines ping and traceroute into one continuously updating tool, showing packet loss and latency at every hop in real time. It's more informative than running the two tools separately.

Use case: A student experiencing intermittent connection drops during a video call uses mtr to watch, in real time, which specific hop along the route is dropping packets.

Structure: mtr [options] host

Option	Purpose
-n	No DNS resolution
-r	Report mode (non-interactive)
-c N	Number of pings per hop

Example:

```
mtr -n -r -c 10 google.com
```

24. ethtool — Query/configure NIC settings

Description: Displays and configures low-level settings of a network interface card (NIC), such as link speed, duplex mode, and driver information.

Use case: A student troubleshooting a surprisingly slow wired connection checks ethtool to confirm whether the NIC negotiated a full 1 Gbps link or fell back to a slower speed.

Structure: ethtool [options] interface

Option	Purpose
ethtool eth0	Show link status, speed, duplex
-i eth0	Show driver info
-S eth0	Show NIC statistics

Example:

```
ethtool eth0
```

25. whois — Domain/IP registration lookup

Description: Queries public registration databases to find ownership and registration details for a domain name or IP address, such as the registrar, owner, and creation date.

Use case: A student researching a suspicious website for a cybersecurity assignment uses whois to find out who registered the domain and when.

Structure: whois domain-or-ip

Example:

```
whois example.com
```

Quick Reference: Legacy → Modern Equivalents

Legacy (deprecated)	Modern replacement
ifconfig	ip addr, ip link
route	ip route
arp	ip neigh
netstat	ss
iptables	nftables (on newer distros)

Tips

- Most of these commands (ip, iptables, tcpdump, nmap with -O) need sudo/root privileges.

- Use `man <command>` or `<command> --help` for the full option list on your specific distro/version.
- Combine `ss/ip` with `watch` for live monitoring, e.g. `watch -n1 ss -tulnp`

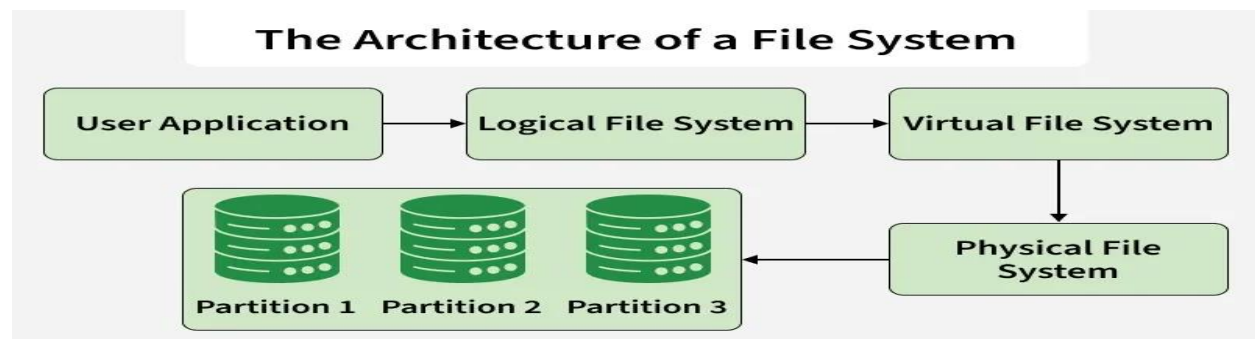
Linux File System

The Linux File System is a structured method of storing and organizing data on a Linux machine. It arranges files in a hierarchical directory format starting from the root directory (/).

- All files and directories in Linux originate from a single root directory (/).
- It follows a hierarchical tree structure, making navigation simple and logical.
- Different directories like /home, /etc, /bin and /var serve specific system purposes.
- Linux supports multiple file system types like ext4, XFS, Btrfs, each offering different features.

Linux File System Structure

The Linux file system architecture is organized into three important layers, each responsible for different functionalities. These layers work together to provide smooth file access, storage and management.



1. Logical File System

Ensures applications interact with files in a simple and consistent way, without worrying about storage details.

- Acts as the interface between user applications and the file system.
- Handles key operations like open, read, write and close.
- Provides security checks, such as permissions and file access control.

2. Virtual File System (VFS)

Makes file operations uniform and compatible, regardless of the underlying file system format.

- Provides a common interface for different file systems (ext4, XFS, FAT32, NTFS, etc.).
- Allows Linux to use multiple file system types at the same time.
- Acts as an abstraction layer, hiding the internal complexities of each file system.

3. Physical File System

Ensures reliable storage, error handling and low-level data management.

- Directly interacts with the hardware and disk storage.
- Manages data blocks, nodes and physical memory allocation.
- Responsible for writing data to the disk and retrieving it efficiently.

Characteristics of a File System

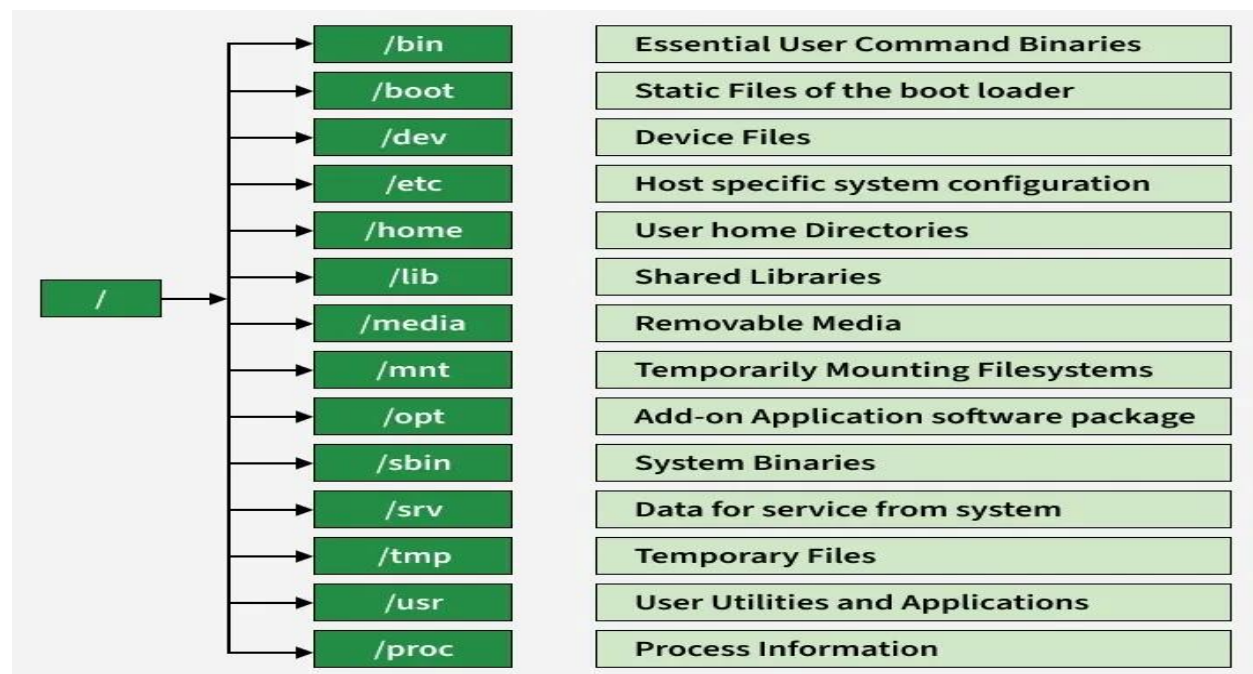
A file system defines the structure, rules and methods for how data is organized, stored, accessed and managed on a storage device.

- **Space Management:** Manages block allocation, free-space tracking and fragmentation control to optimize storage efficiency.
- **Filename:** Enforces naming rules including character sets, length limits and case sensitivity for file identification.
- **Directory:** Implements hierarchical indexing structures to organize and locate files efficiently.
- **Metadata:** Stores file attributes such as ownership, permissions, timestamps, size and file type information.
- **Utilities:** Provides system-level operations for file creation, deletion, backup, recovery and access management.
- **Design:** Defines architectural limits and mechanisms that determine file system scalability, reliability and performance.

Linux File Hierarchy Structure

The Linux File Hierarchy Structure or the Filesystem Hierarchy Standard (FHS) defines the directory structure and directory contents in Unix-like operating systems. It is maintained by the Linux Foundation.

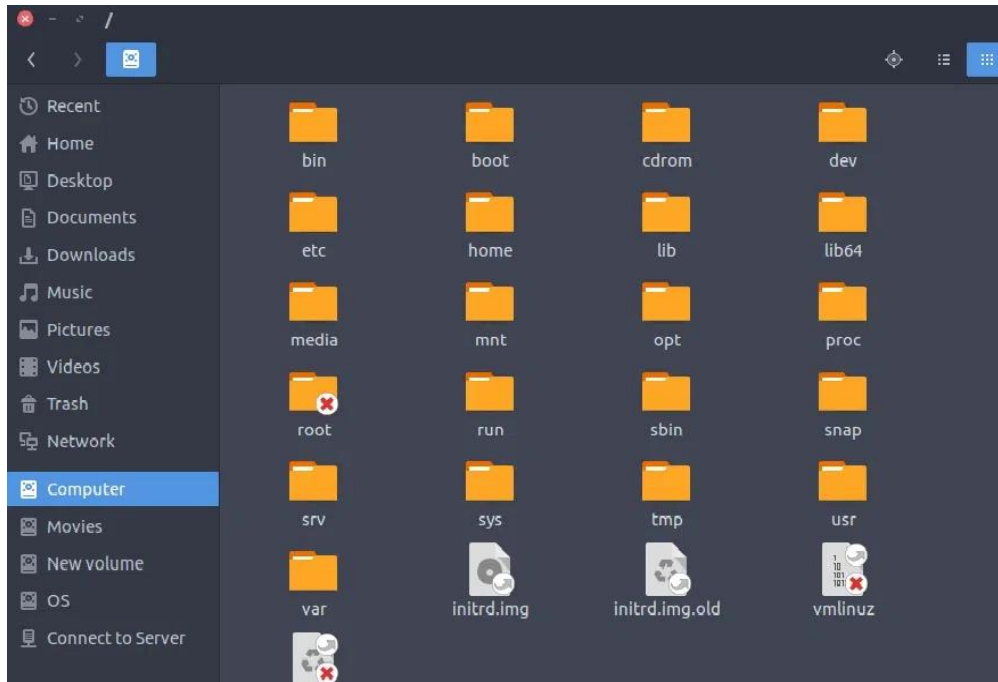
- In the FHS, all files and directories appear under the root directory /, even if they are stored on different physical or virtual devices
- Most of these directories exist in all UNIX operating systems and are generally used in much the same way
- However, the descriptions here are those used specifically for the FHS and are not considered authoritative for platforms other than Linux



1. / (Root)

At the top of every Linux file system is the root directory represented by a forward slash /. It's the base point, and no directory exists above it.

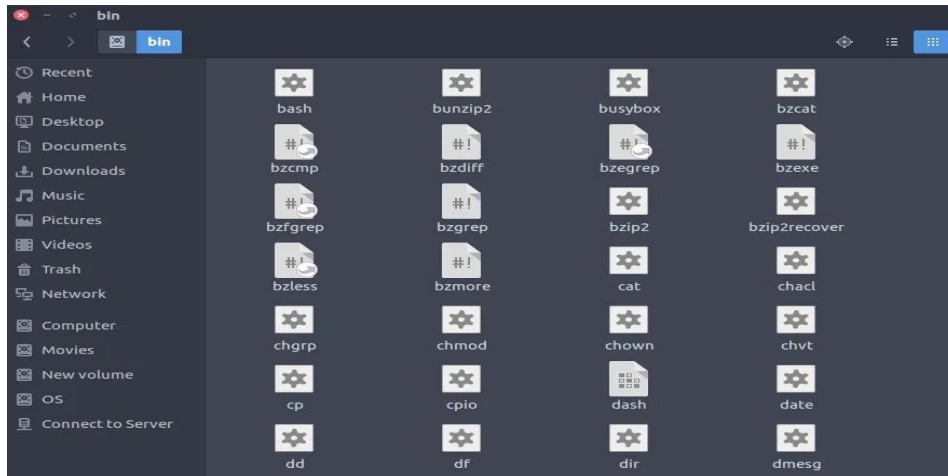
- If you look at the file system graphically, you'll see all other directories branching from this root directory
- Only the root user has permission to modify contents inside this directory.
- Regular users cannot make changes here. For example, if you attempt to create a file in / as a non-root user, you'll encounter a "Permission Denied" error



2. /bin

The /bin directory contains essential commands and binaries needed by all users, including cp, ls, ssh, and kill. These commands are universally available across user types.

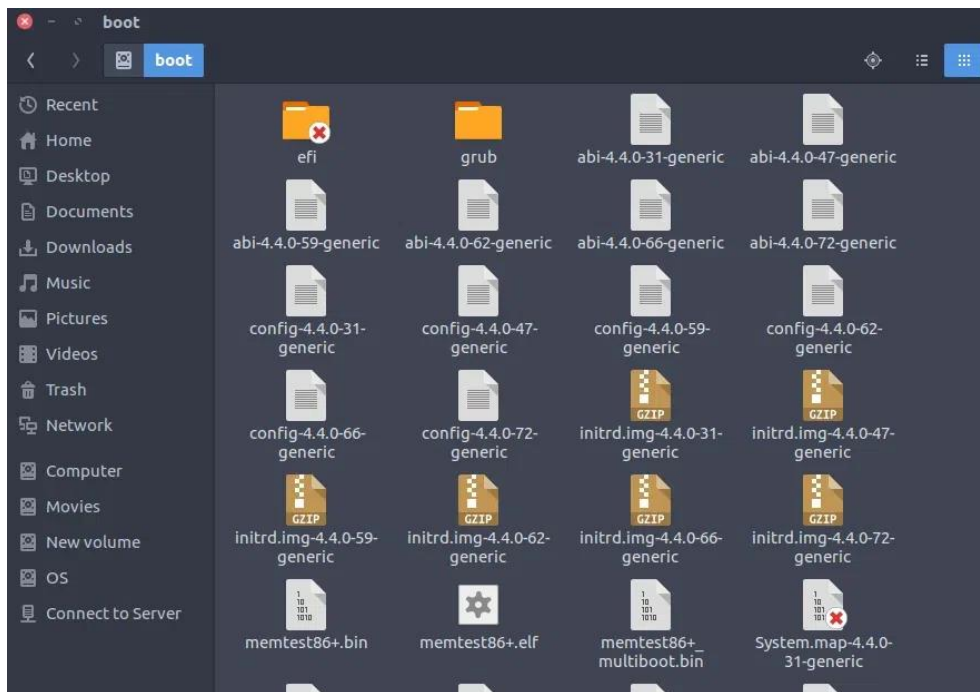
- Contains binary executables
- Common linux commands you need to use in single-user modes are located under this directory
- Commands used by all the users of the system are located here e.g. ps, ls, ping, grep, cp



3. /boot

This directory stores all files required for booting the system. It includes the GRUB bootloader configuration and essential kernel files that are loaded during startup.

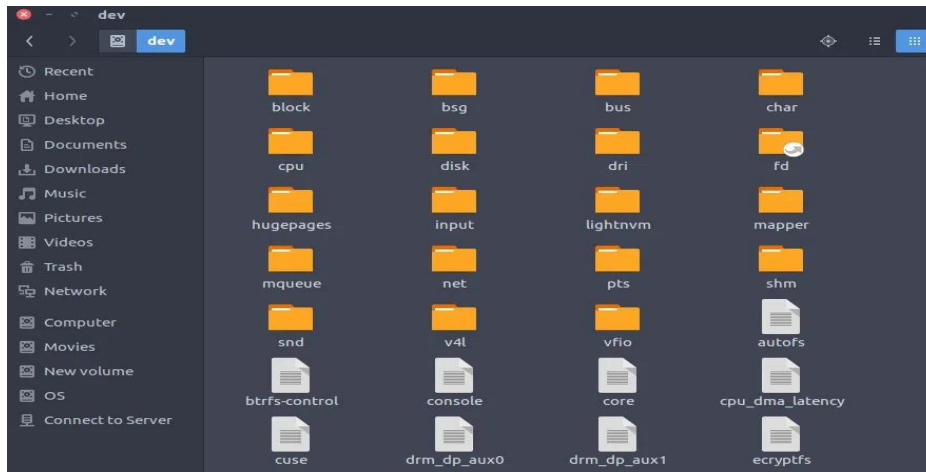
- Kernel initrd, vmlinuz, grub files are located under /boot
- **Example:** initrd.img-2.6.32-24-generic, vmlinuz-2.6.32-24-generic



4. /dev

Device files in Linux are stored in the `/dev` directory. These are special files that act as interfaces between hardware and software. Device files are of two types: block devices (e.g., hard drives) and character devices (e.g., microphones and speakers). Examples include `/dev/sda1` for disk partitions.

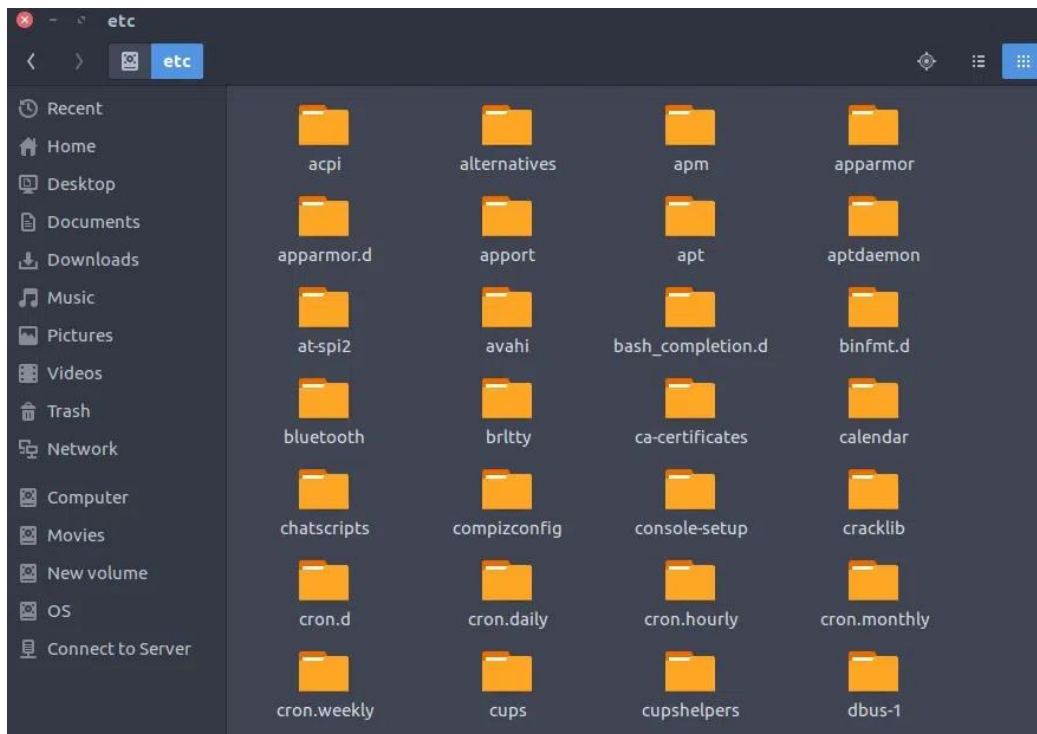
- These include terminal devices, USB, or any device attached to the system
- **Example:** /dev/tty1, /dev/usbmon0



5. /etc

Short for "Editable Text Configuration," /etc contains configuration files for system applications, users, services, and tools or it contains the Host-specific system-wide configuration files. For example, user details like UID and local addresses are defined here.

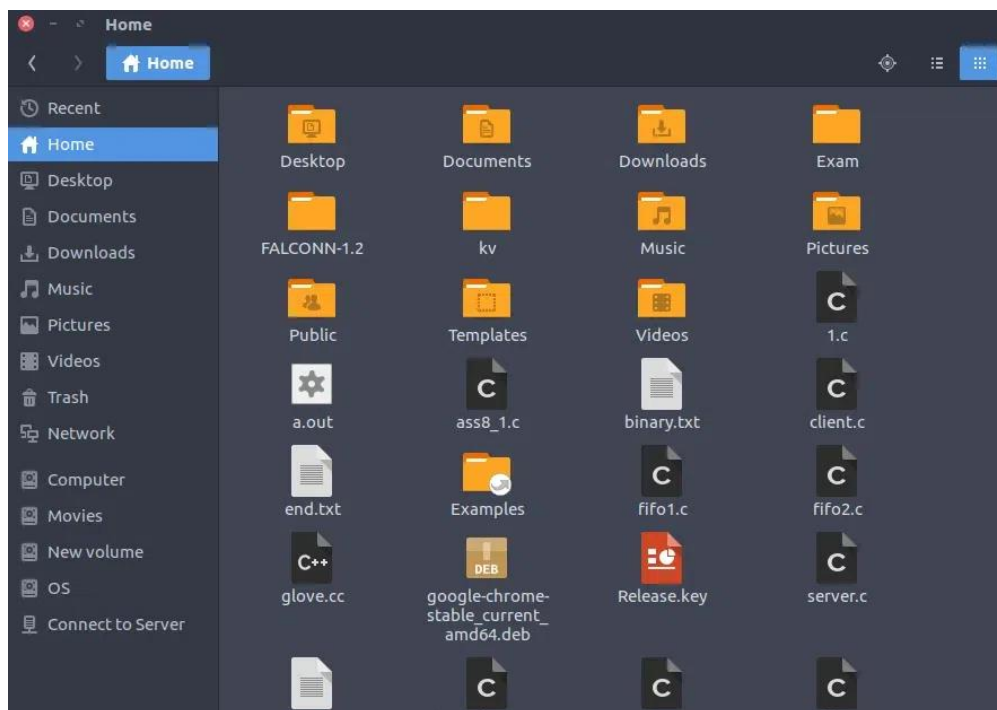
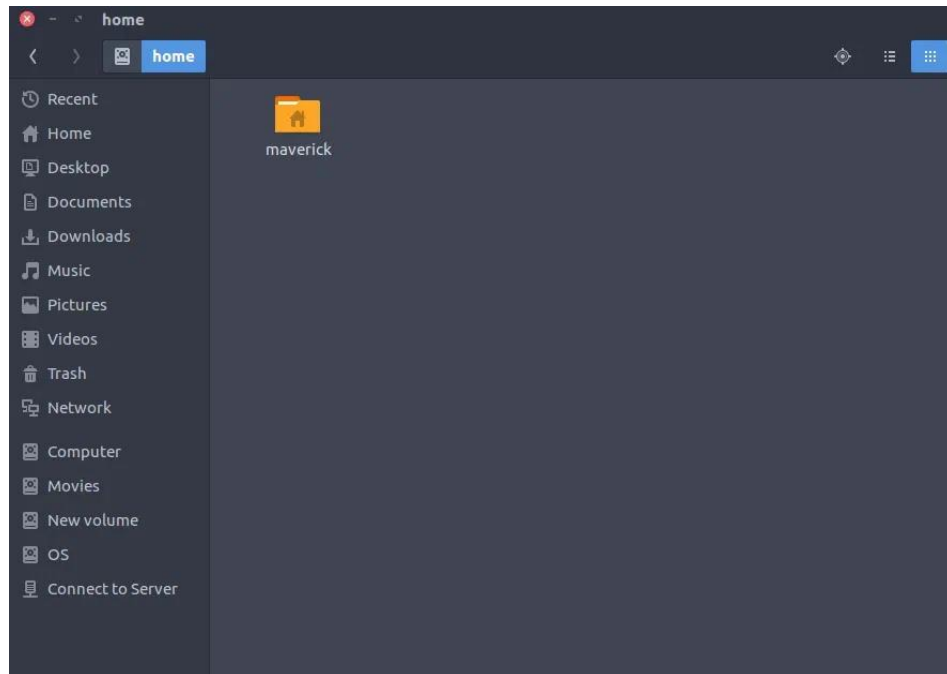
- This also contains startup and shutdown shell scripts used to start/stop individual programs
- **Example:** /etc/resolv.conf, /etc/logrotate.conf



6. /home

Every non-root user has a personal directory inside /home. For example, if your username is raghu, your personal directory would be /home/raghu. Each user can create, delete, or modify files only in their own home directory and access of another user's home depends on security configuration.

- Home directories for all users to store their personal files, containing saved files, personal settings, etc
- **Example:** /home/kishlay, /home/kv



7. /lib

Applications require shared libraries to run, which are stored in /lib. These include dynamic libraries needed during runtime. For example, Apache server libraries are available here.

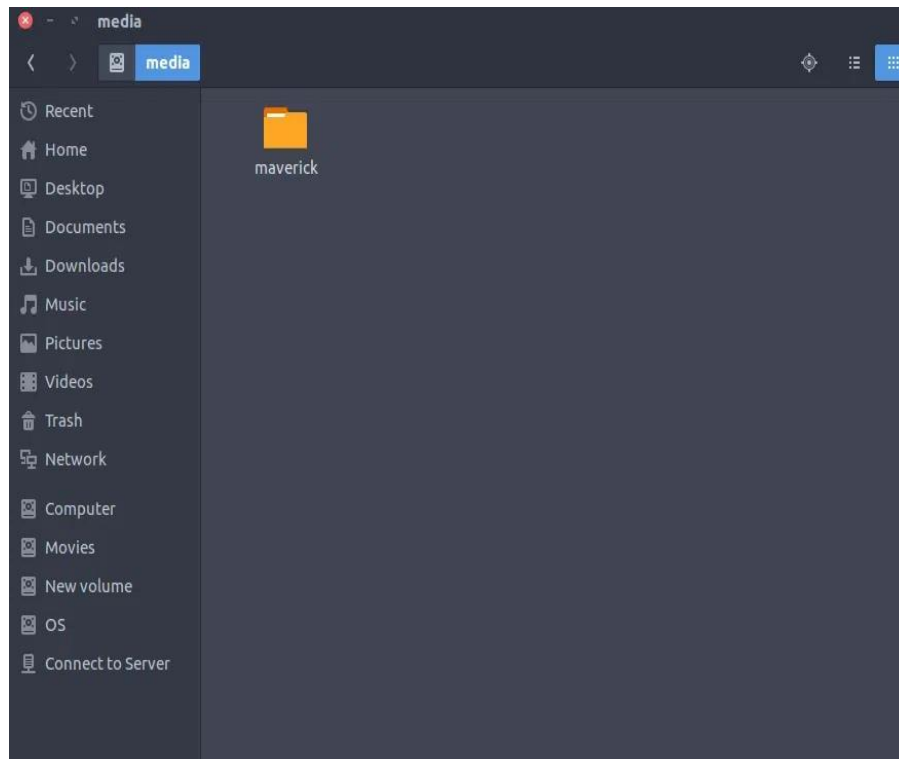
- **Example:** ld-2.11.1.so, libncurses.so.5.7, python3.x



8. /media

Devices like USBs, CDs, and pen drives are mounted under /media. For example, when a CD-ROM is inserted (appeared in FHS-2.3), its details will appear here.

- Temporary mount directory for removable devices
- **Examples:** /media/cdrom for CD-ROM; /media/floppy for floppy drives; /media/cdrecorder for CD writer



9. /mnt

When external drives are connected, they are temporarily mounted in /mnt. This is where their contents become accessible to the system.

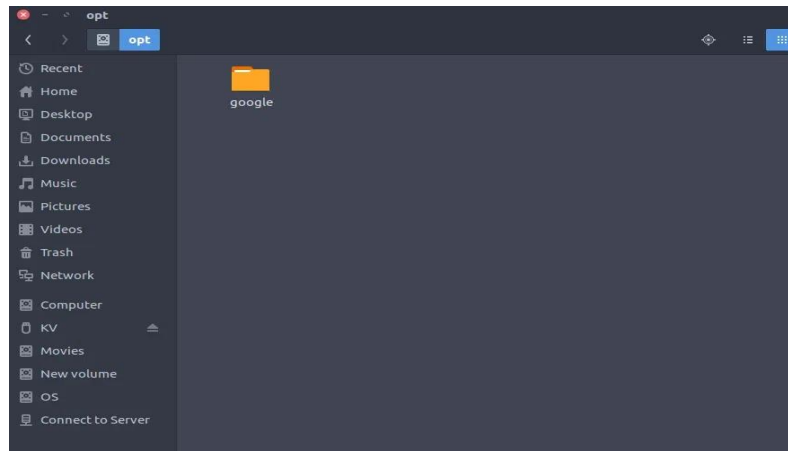
- Temporary mount directory where sysadmins can mount filesystems



10. /opt

Third-party software and packages not part of the default system installation are stored in /opt. It includes their configuration and data files.

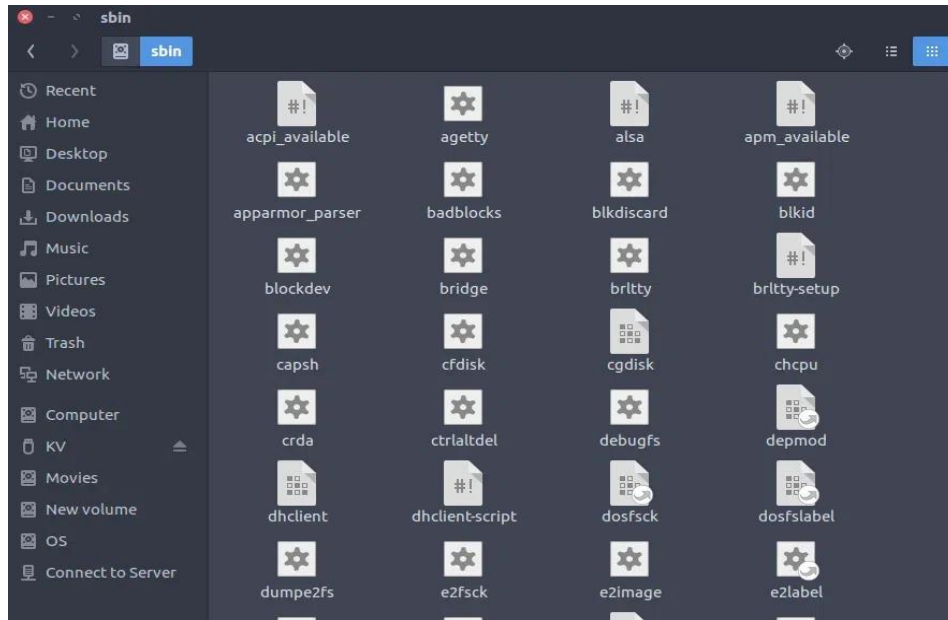
- Contains add-on applications from individual vendors
- Add-on applications should be installed under either /opt/ or /opt/subdirectory



11. /sbin

This directory holds administrative binaries like iptables, firewall management tools, fsck, init, route etc. These binaries are primarily for system administrators and typically require root privileges to execute.

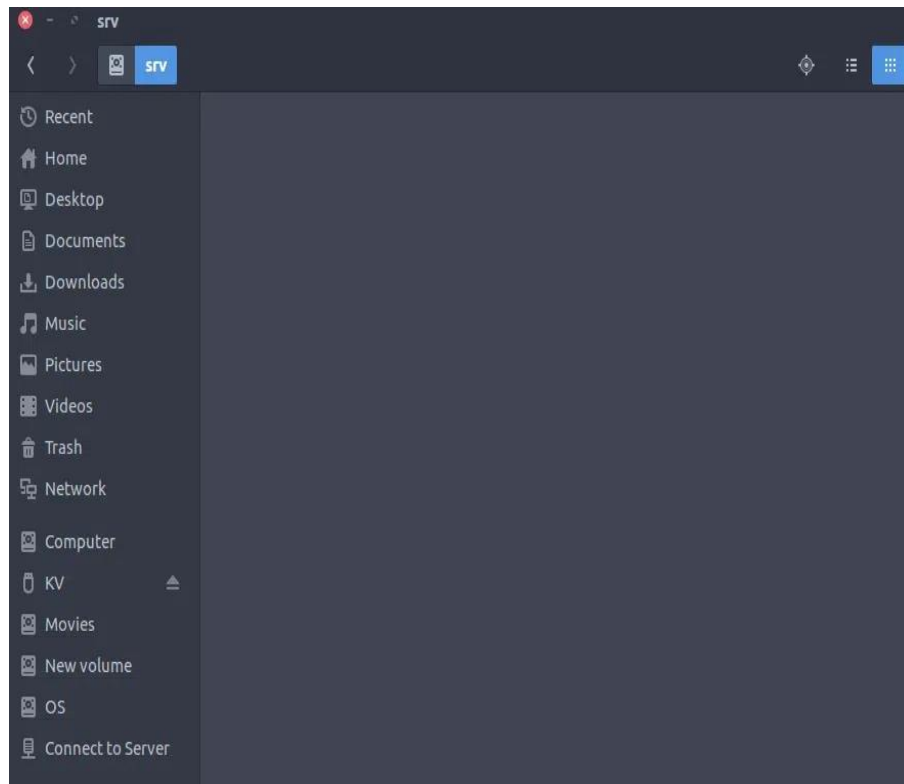
- Just like /bin, /sbin also contains binary executables
- The linux commands located under this directory are used typically by system administrators, for system maintenance purposes
- **Example:** iptables, reboot, fdisk, ifconfig, swapon



12. /srv

Site-specific data served by this system, such as data and scripts for web servers, data offered by FTP servers, and repositories for version control systems.

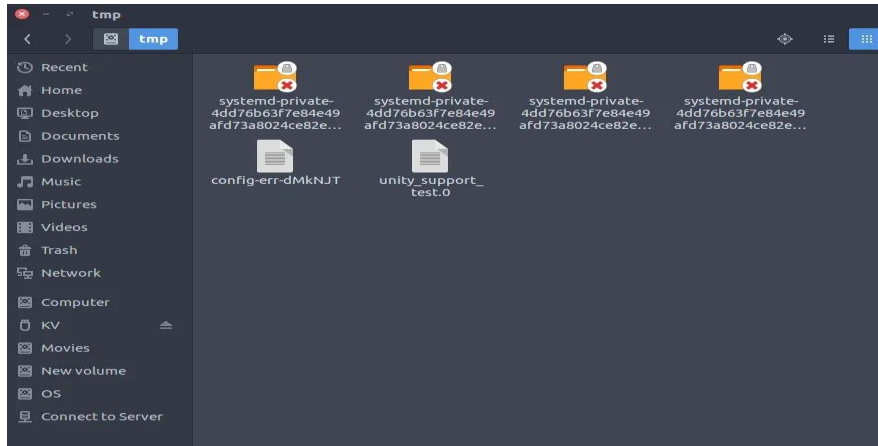
- `srv` stands for service
- Contains server specific services related data
- **Example:** `/srv/cvs` contains CVS related data



13. /tmp

Programs create temporary files during execution, and these are stored in /tmp. These files are deleted automatically after the program finishes or when the system is restarted.

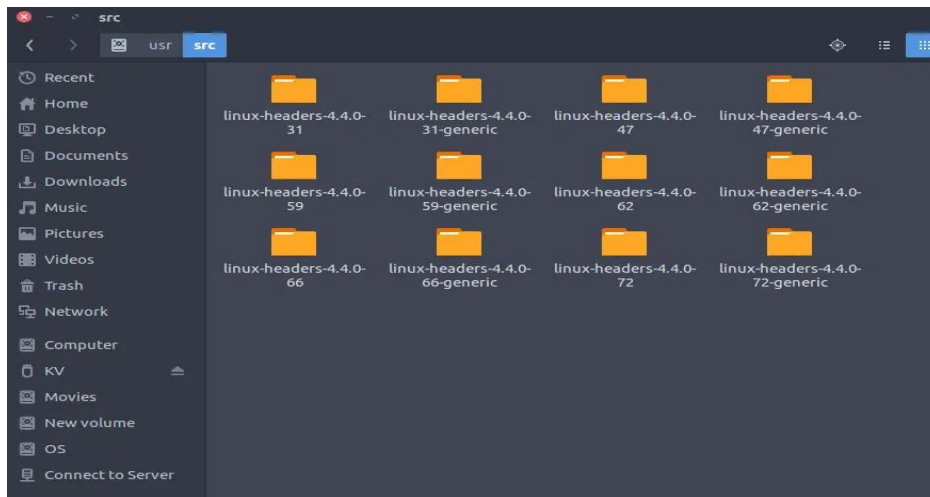
- Directory that contains temporary files created by system and users
- Files under this directory are deleted when the system is rebooted



14. /usr

Secondary hierarchy for read-only user data; contains the majority of (multi-)user utilities and applications.

- Contains binaries, libraries, documentation, and source-code for second level programs
- /usr/bin contains binary files for user programs. If you can't find a user binary under /bin, look under /usr/bin. **For example:** at, awk, cc, less, scp
- /usr/sbin contains binary files for system administrators. If you can't find a system binary under /sbin, look under /usr/sbin. **For example:** atd, cron, sshd, useradd, userdel
- /usr/lib contains libraries for /usr/bin and /usr/sbin
- /usr/local contains user programs that you install from source. For example, when you install apache from source, it goes under /usr/local/apache2
- /usr/src holds the Linux kernel sources, header-files and documentation.



15. /proc

The /proc directory provides detailed information about system processes. Each process is assigned a unique ID and represented as a directory inside /proc. For example, /proc/meminfo gives real-time data about memory usage including total, free, buffer, and cache statistics.

- Contains information about system processes
- This is a pseudo filesystem that contains information about running processes. For example: /proc/{pid} directory contains information about the process with that particular pid
- This is a virtual filesystem with text information about system resources. For example: /proc/uptime

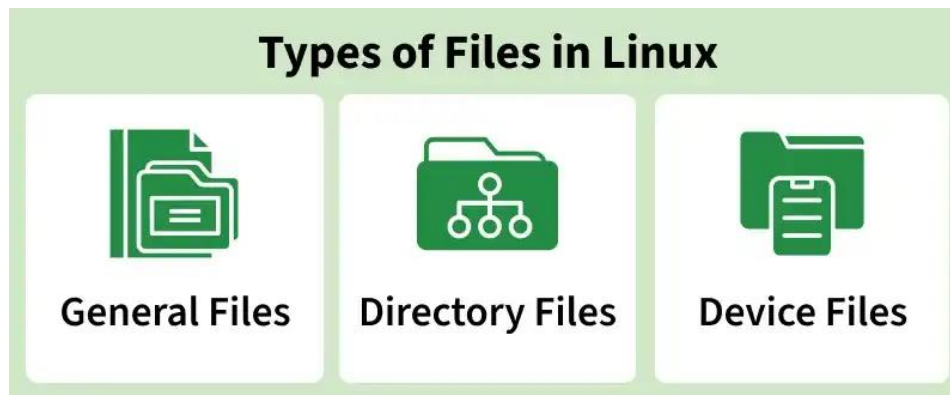
Linux Directory Structure

In Linux, everything is treated as a file even if it is a normal file, a directory, or even a device such as a printer or keyboard. All the directories and files are stored under one root directory which is represented by a forward slash /.

- The Linux directory layout follows the Filesystem Hierarchy Standard (FHS).
- This standard defines how directories are organized and what types of files should be stored in each.
- Since Linux is based on UNIX, it inherits many of UNIX filesystem conventions.
- Similar directory structures are also found in other UNIX like operating systems such as BSD and macOS.

Types of files in the Linux system

Linux categorizes files into different types based on their purpose, such as general data storage, system structure, or device representation.



1. **General Files:** They are also called ordinary files. It may be an image, video, program, or simple text file. These types of files can be in ASCII or Binary format.
2. **Directory Files:** These types of files are a warehouse for other file types. It may be a directory file within a directory (subdirectory).
3. **Device Files:** In a Windows like operating system, devices like CD-ROM and hard drives are represented as drive letters like F: G: H:, whereas in the Linux system, devices are represented as files. For example, /dev/sda1, /dev/sda2, and so on.

We know that in a Windows like operating system, files are stored in different folders on different data drives like C: D: E:, whereas in the Linux/Unix operating system, files are stored in a tree like structure starting with the root directory, as shown in the diagram below.

The Linux/Unix file system hierarchy base begins at the root and everything starts with the root directory.

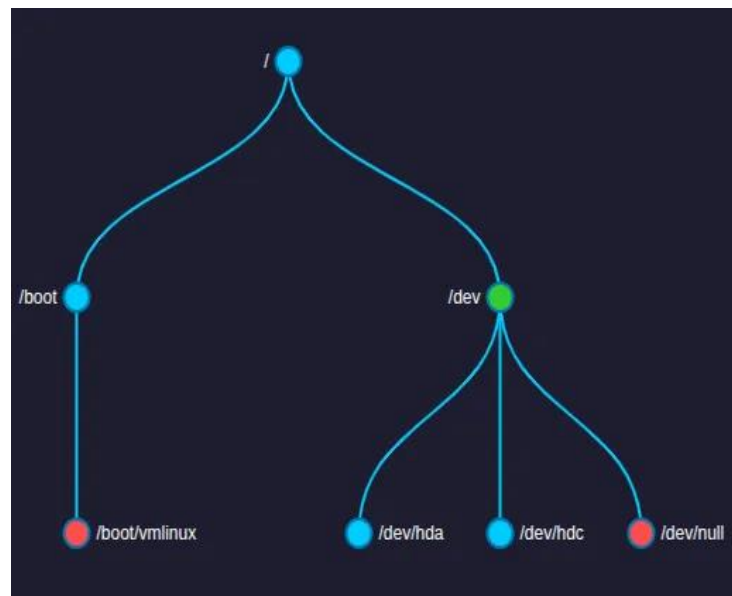
Steps to Check Linux Directories from Root (/)

1. Open the terminal: Launch the terminal from the application menu or by using a keyboard shortcut.
2. Switch to root user: Execute the command `sudo -s` and enter the system password to gain root privileges.
3. Move to the root directory: Change the current working directory to the root (/) directory.
4. List all directories in the root directory: Display all available directories present in the base directory.
5. Review the directory structure: Observe the listed directories such as system, user, configuration, and virtual directories.

```
raghu@ishpc:/$ ls
achal  boot  dataproject  home          jupyterhub.sqlite  lib64      media  proc  sbin  sys  usr
Apps  cdrom  dev          jupyterhub_cookie_secret  lib          libx32     mnt    root  snap tmp  var
bin    data  etc          jupyterhub-proxy.pid     lib32        lost+found  opt    run   srv   user
raghu@ishpc:/$
```

Exploring directories and their usability

In the Linux system some well defined configuration files, binaries, manual pages information files are available for every process.



Linux Kernel File:

- **/boot/vmlinux** : The Linux kernel file.
- **/boot/grub/grub.cfg** : the main generated configuration file actually used at boot time for GRUB

Device Files:

- **/dev/hda** : Device file for the first IDE HDD.
- **/dev/null** : A pseudo device (character device) that discards all data written to it and returns end-of-file (EOF) when read from.

System Configuration Files

System configuration files are essential files stored in the /etc directory that define system behavior, user access, boot settings, network rules, and service management in Linux.

- **/etc/bashrc** : Contains system wide default settings and aliases for the Bash shell.
- **/etc/crontab** : Used to schedule commands or scripts to run automatically at predefined time intervals.
- **/etc/fstab** : Contains details about disk drives and their mount points.
- **/etc/group** : Stores information about system groups and group memberships.
- **/etc/default/grub** : stores user-defined settings (like timeout, default kernel, boot options)
- **/etc/init.d** : Contains service startup and shutdown scripts.
- **/etc/hosts** : Maps IP addresses to corresponding hostnames.
- **/etc/hosts.allow** : Specifies which hosts are allowed to access services on the local machine.
- **/etc/hosts.deny** : Lists hosts that are denied access to services on the local machine.
- **/etc/modules.conf** : Contains configuration settings for system kernel modules.
- **/etc/mtab** : Provides information about currently mounted file systems.
- **/etc/passwd** : Contains user account information such as usernames and user IDs (passwords are stored in the shadow file).
- **/etc/profile** : Defines system wide environment settings for the Bash shell.
- **/etc/profile.d** : Contains additional scripts executed automatically after user login.
- **/etc/resolv.conf** : Stores DNS configuration used by the system.
- **/etc/security** : Contains security related configuration files, including login access controls.
- **/etc/termcap** : ASCII file defining behavior and capabilities of different terminal types.
- **/etc/X11** : Directory containing configuration files for the X Window System.

User Related Files

A top-level directory in Linux that contains user-space programs, libraries, documentation, and shared data. Below are some examples :

- **/usr/bin** : Contains most of the user level executable programs and commands.
- **/usr/include** : Stores standard header files used by C and other programming languages.
- **/usr/share** : Contains architecture independent, shareable data such as documentation and text files.
- **/usr/lib** : Holds object files and shared libraries required by programs in /usr/bin.
- **/usr/sbin** : Contains administrative commands used by the system administrator (superuser).

Virtual and Pseudo Process Related Files

These /proc files provide real time system and process information generated dynamically by the Linux kernel.

- **/proc/cpuinfo** : Provides detailed information about the CPU, such as processor type, model, and speed.
- **/proc/filesystems** : Displays the file systems supported and currently available in the system.
- **/proc/interrupts** : Shows information about the number of interrupts handled by each IRQ.

- **/proc/ioproports** : Contains details of input and output addresses used by hardware devices.
- **/proc/meminfo** : Reports detailed information about system memory usage.
- **/proc/modules** : Lists the kernel modules that are currently loaded in the system.
- **/proc/mounts** : Displays information about all mounted file systems.
- **/proc/stat** : Provides detailed statistics about system activity and performance.
- **/proc/swaps** : Contains information about swap space and swap files in use.
- **/proc/version** : Contains the Linux version information.

Log Files

These files record crucial system events, logins, and activity history for monitoring and troubleshooting.

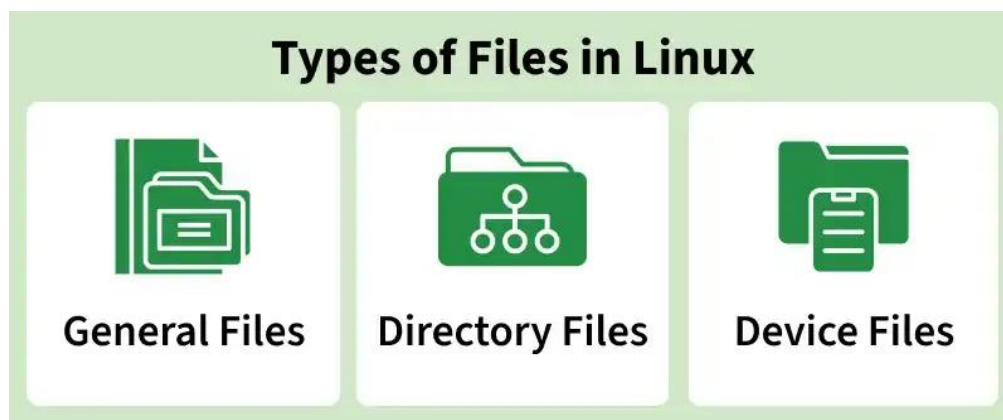
- **/var/log/lastlog** : Stores information about the last login of each user.
- **/var/log/messages** : Contains general system activity and global log messages, though on modern systemd-based systems logging may instead be handled by journald
- **/var/log/wtmp** : Maintains a history of user login and logout sessions.

File Management in Linux

File management in Linux involves handling files and directories through various operations such as creation, modification, organization and access control within the filesystem.

- Linux treats everything as a file, including devices and system configurations.
- Ensures efficient data organization and accessibility.
- Common commands include cp, mv, rm, ls and cat.

Linux categorizes files into three main types, each serving a specific purpose in the system:



1. General Files

These are the most common file types that store user data such as text files, images, and binaries.

- Represent regular data like documents, programs, or scripts.
- Form the majority of files in Linux/UNIX systems.
- May contain human-readable text (ASCII), executable binaries, or program data.

2. Directories:

These act as containers that organize files and other directories hierarchically.

- Similar to folders in Windows.
- Store lists of file names and their related metadata.

Important directories include:

- `/`: Root directory (base of the system)
- `/home/`: User home directories
- `/bin/`: Essential user binaries
- `/boot/`: Static boot files

3. Device Files:

These files represent hardware devices and handle input/output (I/O) operations.

- Used to interact with physical devices like printers, disks, or terminals.
- Found mostly in the `/dev/` directory.
- Allow the operating system to treat hardware as if it were a regular file.